

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	Manoj M	Reg. No.:	192511060
Section / Batch:	CSE	Date:	29.07.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Q51. Debugging Selection Sort Code (Swap Executed Inside Inner Loop)

The following Selection Sort implementation performs swapping inside the inner loop before the minimum element for the pass is fully identified, which causes unstable and incorrect partial ordering. Carefully analyze the execution flow and explain why repeated early swaps break the intended behavior of Selection Sort. Apply systematic debugging to ensure the minimum element is selected completely before any swap occurs. Optimize the implementation by performing a swap only once per outer pass when necessary. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i+1, n):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i] # ERROR  
    return arr
```

Code of Conduct

I Manoj M(192511060) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1) Debugging selection sort code.

1. analytical problem solving - written.

Problem Analysis

* The given selection sort implementation contains a logical error where the swap operation is executed inside the inner loop.

* Since, swapping occurs immediately after every comparison, the algorithm keeps moving elements before finding the Actual minimal value.

Why the error occurs

1. The min elements has not yet been fully identified.

2. Swapping during every iteration changes the array repeatedly.

3. The algorithm performs many unnecessary swaps.

4. The final sorted order may become incorrect for several repeats.

Space - complexity

$O(1) \rightarrow$ In-place sorting.

2. Scenario-Based – Execution Scenario

A smart grid monitoring system stores electricity consumption readings collected from different houses. Before generating reports, the readings must be arranged in ascending order using Selection Sort.

Input Execution Pass 1

Minimum = 11

Pass 2

Minimum = 12

Pass 3

Minimum = 22

Pass 4

Minimum already at correct position.

Final Output

3. Debugging & Optimisation Task – Execution

Buggy Code

Errors Identified

1. Swap statement is placed inside the inner loop.
 2. Swapping occurs before the minimum element is completely identified.
 3. Multiple unnecessary swaps are performed.
 4. Algorithm does not follow the Selection Sort principle.
 5. Sorting may produce incorrect results.
-

Optimised Code

Optimisation Performed

- Moved the swap operation **outside the inner loop**.
- Swap is executed **only once per outer pass**.
- Added the condition if `min_idx != i` to avoid unnecessary swaps.

- Reduced the total number of swap operations.
- Preserved the standard Selection Sort algorithm.

Complexity Analysis

Parameter	Complexity
-----------	------------

Best Case	$O(n^2)$
-----------	----------

Average Case	$O(n^2)$
--------------	----------

Worst Case	$O(n^2)$
------------	----------

Space Complexity	$O(1)$
------------------	--------

Number of Swaps At most **$n - 1$**

Final Conclusion

The original implementation failed because it performed swaps before identifying the minimum element of each pass. After moving the swap statement outside the inner loop and swapping only when necessary, the algorithm correctly follows the Selection Sort technique, reduces unnecessary operations, and produces the correct sorted output.